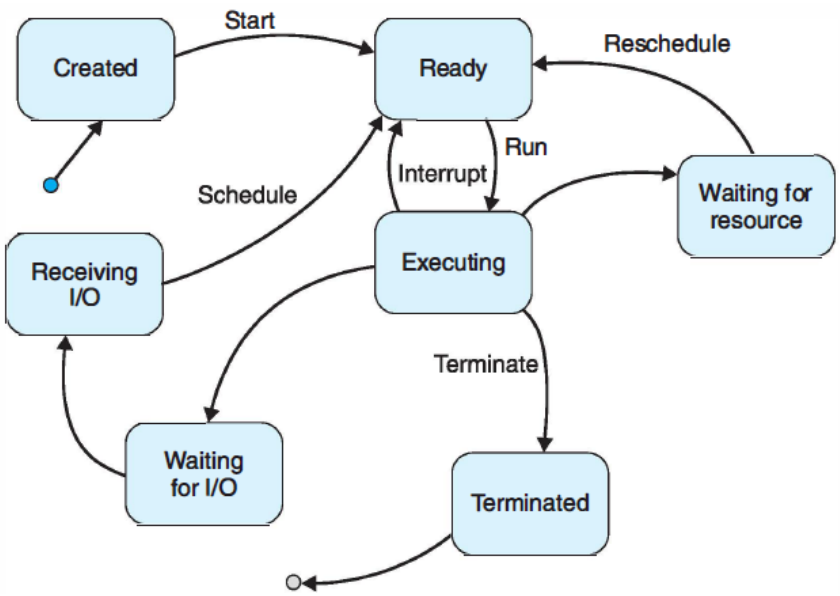
Process management is one of the major functions of the operating system; it involves creating processes and controlling their execution. In most operating systems, several processes are stored in memory at the same time and the OS manages the sharing of one or more processors (CPUs) and other resources among the various processes. This technique implemented in the operating system is called multiprogramming. The number of processes in memory is called the degree of multiprogramming. One of the requirements of multiprogramming is that the OS must allocate the CPUs and other system devices to the various processes in such a way that the CPUs and other devices are maintained busy for the longest total time interval possible, thus minimizing the idle time of these devices. If there is only one CPU in the computer system, then only one process can be in execution at any given time. Some other processes are ready and waiting for CPU service while other processes are waiting or receiving I/ 0 service.

## PROCESS STATES

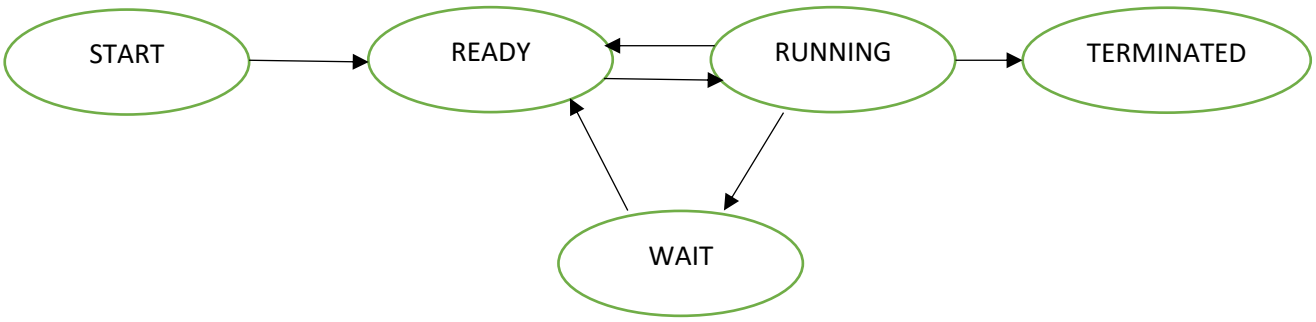
The process manager controls execution of the various processes in the system. Processes change from one state to another during their lifetime in the system. The state changes are really controlled by the OS. Normally, a process spends a finite time in any of its states. When a job arrives, and if there are sufficient resources (such as memory) available, a corresponding process is created. A state diagram represents the various states of a process and the possible transitions in the behavior of a process. Below is the state diagram of a typical process.




When a job arrives, and if there are sufficient resources (such as memory) available, a corresponding process is created. After being created, the process becomes ready and waits for CPU service. It eventually receives this service, then waits for I/O service, and at some later time receives I/O service. The process then goes back to the ready state to wait for more CPU service. After satisfying all service requests, the process terminates.

In general, a process can have one of the following five states at a given time:

| S/N | STATE | DESCRIPTION |
|---|---|---|
| 1 | START | This is the initial state when a process is first started or created |
| 2 | READY | The process is waiting to be assigned a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. A process may come into this state after the START state or while it is running (i.e. when it is interrupted by the scheduler to assign the CPU to some other process. |
| 3 | RUNNING | Once the process has been assigned to a processor, the process state is set to running and the processor executes its instructions. |
| 4 | WAITING | Process moves into the waiting state if it needs to wait for a resource, such as waiting for a user input, or waiting for a file to become available. |
| 5 | TERMINATED or EXIT | Once the process finishes its execution or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from the main memory. |




## INTERRUPTING PROCESSES

When a process is executing, it is in the running state. At any time instant, the OS may interrupt the process and switch to the next process that is waiting for CPU service. There are three basic reasons for interrupting this process:

- The running process needs an I/O operation, such as reading data from a file. In this case, the process invokes a system function of the OS and suspends itself. When the I/O operation is complete, the process is reactivated and placed again in the ready queue to wait for CPU service. In the meantime, the OS has switched to the next process and starts to execute it.

- The running process is interrupted by a timer under OS control. The process returns to its ready state in the ready queue to wait for CPU service. The OS removes the next process from the ready queue and switches to this process.

- A high priority process arrives or is made to transition to the ready state, and the OS interrupts the running process if it has a lower priority. The interrupted process is normally returned to the ready queue to wait for CPU service.

The selection of the next process to select from the ready queue is carried out by the scheduler. The allocation of the CPU to this new process is carried out by the dispatcher, another component of the OS.

## PROCESS DESCRIPTOR / PROCESS CONTROL BLOCK

For every new process in the system, a data structure called the process descriptor, also known as a process control block (PCB), is created. A Process Control Block is a data structure maintained by the operating system for every process. It is on this data structure that the OS stores all data about a process. The various queues that the OS manipulates thus contain process descriptors or pointers to the process descriptors, not actual processes. A process descriptor represents a process in the system and when the process terminates, its descriptor is normally destroyed. A process descriptor includes several data items or fields and is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process as listed below in the table:

| INFORMATION | DESCRIPTION |
|---|---|
| Process State | The current state of the process i.e. whether it is ready, running, waiting etc. |
| Process Privileges | This is required to allow or disallow access to system resources. |
| Process ID | Unique identification for each of the process in the operating system. |
| Pointer | A pointer to parent process |
| Program Counter | Program counter is a pointer to the address of the next instruction to be executed for this process. |
| CPU Registers | Various CPU registers where process need to be stored for execution for the running state |
| CPU Scheduling Information | Process priority and other scheduling information which is required to schedule the process |
| Accounting Information | This includes the amount of CPU used for process execution, time limits, execution ID etc. |
| I/O status information | This includes a list of I/O devices allocated to the process |

The architecture of a PCB is completely dependent on the operating system and may contain different information in different operating systems. The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminates.

## PROCESSOR SCHEDULING

The main goal of scheduling is the sharing of one or more processors among the processes in the ready queue. The critical activities that the operating system scheduler must carry out are the ordering of the allocation and deallocation of the processor(s) to the various processes and threads, one at a time. The other activity is deciding when to deallocate a processor from a process. These activities must be carried out in a way that meets the performance objectives of the system. Since the processors are the most important resource in a computer system, the operating system must manage it in the most efficient and effective manner. To carry out multiprogramming, the operating system provides scheduling policies and mechanisms. A scheduling policy defines the order in which processes are selected from the ready queue for processing. The scheduling mechanism decides when and how to carry out the context switch to the selected process-Le., the deallocation of a processor from the current process and the allocation of the processor to the selected process.

## GENERAL TYPES OF SCHEDULING

**Long-term scheduling**: The operating system decides to create a new processes from the jobs waiting in the input queue and loads them into memory for execution. This decision controls the degree of multiprogramming. The operating system may decide to create a new process when a currently executing process terminates or when it needs to increase or limit the degree of multiprogramming. The selection of which job to select from the waiting list is based on several criteria. The primary objective here is to provide a balanced mix of jobs such as I/O bound and processor bound. Long-term schedulers are also called job schedulers.

**Short-term scheduling**: The operating system decides which process to execute next. In other words, the operating system decides when and to which process the CPU will be allocated next. This type of scheduling is often called CPU scheduling. The main objective is to increase system performance in accordance with the chosen set of criteria. Short-term schedulers are also known as dispatchers. Short-term schedulers are faster than long-term schedulers.

**Medium-term scheduling**: The operating system decides when and which process to swap out or swap in from or to memory. This also controls the degree of multiprogramming.

## COMPARISON AMONG SCHEDULERS

| Long-Term Scheduler | Short-Term Scheduler | Medium-Term Scheduler |
|---|---|---|
| It is a job scheduler | It is a CPU scheduler | It is a process swapping scheduler |
| Speed is lesser than short term scheduler | Speed is fastest among the other two | Speed is in between both short and long term schedulers |
| It controls the degree of multiprogramming | It provides lesser control over degree of multiprogramming | It reduces the degree of multiprogramming |
| It selects processes from the pool and loads them into memory for execution | It selects those processes which are ready to execute | It can re-introduce the process into memory and execution can be continued |

## CPU SCHEDULING POLICIES

There are two general categories of CPU scheduling policies:

**Non preemptive scheduling**: Here, a process that is executing will continue until completion of its CPU burst. The process will then change to its wait state for I/O service, or terminate (change to the terminate state) and exit the system.

**Preemptive scheduling**: Here, the process that is executing may be interrupted before completion of its current CPU burst and moved back to the ready queue.

## OPERATING SYSTEM PERFORMANCE MEASURES IN SCHEDULING

CPU utilization: The proportion of time that the CPU spends executing processes.

**Throughput**: The total number of processes that are executed and completed during some observation period.

**Process average waiting time**: The average of the waiting intervals of all processes that are completed.

**Average turnaround time**: The average of the intervals from arrival until completion for all processes.

**Average response time**: The average of the intervals from the time a process sends a command or request to the operating system until a response is received, for all processes. This metric is used mainly in interactive systems.

**Fairness**: A metric that indicates if all processes are treated in a similar manner.

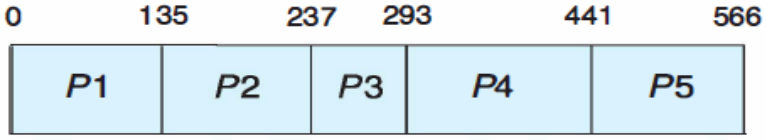
## CPU SCHEDULING POLICIES/ALGORITHMS

**First-Come-First-Served**: First-come-first-served (FCFS) is the most basic and simplest scheduling policy; it is implemented with a first-in-first-out (FIFO) queue. The arrival order of the processes into the ready queue determines the selection of the next process to service. In other words, the process that has been waiting the longest period in the ready queue is selected next for execution. This policy is normally single class and non-preemptive.

Let five processes arrive to the ready queue in the order Pl, P2, P3, P4, and P5 at time 0, 1, 2, 3,4 respectively. The table below shows the CPU bursts for the processes; assume that the time units are in microseconds.

| Process | CPU BURST |
|---|---|
| P1 | 135 |
| P2 | 102 |
| P3 | 56 |
| P4 | 148 |
| P5 | 125 |

The Gantt chart (or execution diagram) in the figure below shows the starting times, execution intervals, and completion times for the five processes.




For this problem, process Pl starts immediately, so its waiting time is zero. Process P2 waits until Pl completes, process P3 waits until P2 completes, process P4 waits until P3 completes, and process P5 waits until P4 completes.

| Process | Arrival Time | CPU Burst/Execute Time | Start/Service Time | Wait Time | Turnaround Time |
|---|---|---|---|---|---|
| P1 | 0 | 135 | 0 | 0 | 135 |
| P2 | 1 | 102 | 135 | 135 | 236 |
| P3 | 2 | 56 | 237 | 235 | 291 |
| P4 | 3 | 148 | 293 | 290 | 438 |
| P5 | 4 | 125 | 441 | 437 | 562 |

Wait Time = Service Time – Arrival Time
Turnaround Time = Completion Time – Arrival Time
Normalized Turnaround Time = Turnaround Time/CPU Burst Time
Average Wait Time

Average Turnaround Time

**SHORTEST JOB FIRST/SHORTEST JOB NEXT/SHORTEST PROCESS NEXT**: This is a scheduling policy in which the scheduler selects from the ready queue the process with the shortest CPU service time interval (burst). This scheduling policy can be considered multiclass because the scheduler gives preference to the group of processes with the shortest CPU burst. It is also a non-preemptive scheduling policy. An internal priority is used for each group or class of processes. The operating system assigns a higher priority to the group of processes that has the shortest CPU service time interval (or CPU burst). In other words, the scheduler gives preference to the groups of processes with shorter CPU bursts over other groups of processes. Shortest process next scheduling is provably optimal because it results in the minimum wait time for processes. However, when processes with shorter service periods continue arriving into the ready queue, the processes with longer service demand periods may be left waiting indefinitely. This situation is known as starvation.

| Process | CPU BURST |
|---|---|
| P1 | 135 |
| P2 | 102 |
| P3 | 56 |
| P4 | 148 |
| P5 | 125 |

| Process | Arrival Time | CPU Burst/Execute Time | Start/Service Time | Wait Time | Turnaround Time |
|---|---|---|---|---|---|
| P1 | 0 | 135 | 0 | 0 | 135 |
| P2 | 1 | 102 | 191 | 190 | 292 |
| P3 | 2 | 56 | 135 | 133 | 189 |
| P4 | 3 | 148 | 418 | 415 | 563 |
| P5 | 4 | 125 | 293 | 289 | 414 |